# Smart-Camera-K15-SL — Control4 Driver

## Overview

Complete Control4 driver for Slomins K15-SL IP Camera with full API integration, authentication, streaming, PTZ support, and real-time MQTT event detection.

- **Version:** 10
- **Package:** `Smart-Camera-K15-SL.c4z` (22.1 KB)
- **Minimum Control4 OS:** 3.3.2+

## Features

### Full API Integration

- Initialize camera and retrieve public key
- User authentication with RSA-OAEP encryption
- Token management (temporary and exchange tokens)
- Device listing

### Streaming Support

- RTSP main stream (high quality)
- RTSP sub stream (low quality)
- Dynamic URL generation

### Camera Functions

- Snapshot URL generation
- PTZ controls (Pan, Tilt, Zoom)
- Preset management

### Security

- RSA-OAEP + SHA256 encryption using `C4:Crypto()`

- HMAC-SHA256 signatures for API requests
- Secure token storage

### Real-Time Event Detection (MQTT)

- Motion, human, face, and stranger detection
- Doorbell ring notifications with snapshot
- Camera online/offline monitoring

## Requirements

- **Control4 Composer Pro** (appropriate version for your platform)
- **Control4 OS** 3.3.2 or newer (for RSA encryption support)
- **Network Access** to API endpoint and camera
- **Camera** accessible on LAN

## Installation

### 1. Install Driver Package

1. Open **Control4 Composer Pro**
2. Go to **Drivers** menu → **Add Driver** → **Install From File**
3. Select `Smart-Camera-K15-SL-v0.1.0.c4z`
4. Click **Install**
5. Driver will appear in available drivers list

### 2. Add Device to Project

1. Go to **Project** view
2. Select a room
3. Click **Add Device**
4. Search for "Slomins K15-SL"
5. Add to room

### 3. Configure Properties

**API Configuration:**

- **Base API URL:** `https://api.arpha-tech.com`
- **Account:** Your email or phone number

**Camera Configuration:**

- **IP Address:** Camera IP (e.g., 192.168.1.100)
- **HTTP Port:** 80 (default)
- **RTSP Port:** 554 (default)
- **Username:** Camera username
- **Password:** Camera password

## Quick Start Guide

```
1. Execute "Initialize"
   → Gets RSA public key from API
   → Stores in "Public Key" property

2. Execute "Login/Register"
   → Encrypts credentials with RSA-OAEP
   → Authenticates with API
   → Stores Auth Token

3. Execute "Get Devices"
   → Lists all devices for user
   → View results in Lua Output

4. Execute "Test Main Stream"
   → Generates RTSP URL for streaming

5. Execute "Get Snapshot URL"
   → Generates HTTP snapshot URL
```

---

## Real-Time Event Detection (MQTT)

The driver supports **MQTT-based real-time event streaming over SSL (port 8884)** and provides notifications for the following events:

- Doorbell ring notifications with snapshot

- Motion detection with snapshot attachment
- Human detection
- Face detection
- Stranger detection
- Camera online/offline status monitoring

# MQTT Configuration

### Default Behavior

MQTT is **enabled automatically by the driver after authentication**. The driver will automatically:

1. Enable MQTT
2. Fetch MQTT credentials
3. Connect to the MQTT broker
4. Subscribe to camera event topics

You normally **do not need to enable MQTT manually**.

---

# CldBus App Motion Detection Settings

To receive **Motion Detection** and **Human Detection** events, detection must be enabled in the **CldBus mobile app**.

### Steps

1. Open the **CldBus App**
2. Select your **camera device**
3. Tap **Settings**
4. Navigate to `Settings → Motion Detection`
5. Under **Detection Type**, select one of the following:

| Detection Type | Description |
|---|---|
| **All Detections** | All movement events will trigger notifications. This includes general motion detection. |

| Detection Type | Description |
| --- | --- |
| **Human Detection** | Only human movement will trigger events. Non-human motion will be ignored. |

---

# Push Notification Configuration

## 1. Create Push Notification

1. Open **Composer Pro**
2. Navigate to `Agents → Push Notification`
3. Click **Add Notification** then enter a name for the notification

Configure the following:

| Setting | Value |
| --- | --- |
| Category | Cameras |
| Severity | Info / Critical |
| Subject | Camera Event |

Click **Save**.

## 2. Enable Snapshot Attachment

Edit the created notification and set:

```
Attachment Type = Snapshot URL
```

This allows push notifications to include the **camera snapshot image**.

## 3. Map Push Notifications in Programming

1. In **Composer Pro**, open **Programming**
2. Select the **Camera Driver** from the device list
3. In the **Events** section, choose the event to trigger (e.g. **Motion Detected**)
4. On the right-side panel, click **Push Notifications**

5. From the dropdown, select the notification you created earlier
6. Drag and drop the notification into the programming area **or double-click the green arrow** to add it

Example:

```
WHEN Motion Detected → THEN Send Push Notification
```

### Notification Flow

```
Camera Event (MQTT)
        ↓
Driver Receives Event
        ↓
Driver Processes Event
        ↓
Control4 Event Triggered
        ↓
Push Notification Agent
        ↓
Mobile Notification Sent
        ↓
Snapshot Image Attached
```

---

## Available Actions

All actions accessible via: **Device → Actions → Select action → Execute**

### Authentication & API

- **Initialize** — Get RSA public key from API
- **Login/Register** — Authenticate user and get auth token
- **Get Temp Token** — Get temporary token (300 seconds)
- **Get Exchange Token** — Exchange temp token for identity token
- **Get Devices** — List all user devices

### Streaming & Snapshots

- **Test Main Stream** — Generate high quality RTSP URL

- **Test Sub Stream** — Generate low quality RTSP URL
- **Test Snapshot** — Generate HTTP snapshot URL

### PTZ Controls

- **PTZ Up** — Move camera up
- **PTZ Down** — Move camera down
- **PTZ Left** — Move camera left
- **PTZ Right** — Move camera right

---

## API Functions

### 1. Initialize Camera

**Command:** `INITIALIZE_CAMERA`
**Endpoint:** `POST /api/v3/openapi/init`
**Purpose:** Retrieve RSA public key for encryption

What it does:

- Generates client ID and request ID
- Creates HMAC-SHA256 signature
- Requests public key from API
- Stores key in "Public Key" property

**Expected Output:**

```
Camera initialization succeeded
Received public key: MIGfMA...
Public key stored successfully
```

### 2. Login/Register

**Command:** `LOGIN_OR_REGISTER`
**Endpoint:** `POST /api/v3/openapi/auth/login-or-register`
**Purpose:** Authenticate user with encrypted credentials

**Prerequisites:** Must run Initialize first (needs public key). Account must be set in properties.

Encryption Process:

```
1. Create crypto object: C4:Crypto("RSA")
2. Import public key
3. Encrypt JSON: {"country_code":"US","account":"user@email.com"}
4. Generate HMAC-SHA256 signature
5. Send to API
```

**Expected Output:**

```
Using C4:Crypto for RSA-OAEP encryption...
Crypto object created successfully
Public key imported successfully
Encrypting data with RSA-OAEP + SHA256...
Encryption successful!
Login/Register succeeded
Auth token stored
User ID: 12345
```

## 3. Get Temp Token

**Command:** `GET_TEMP_TOKEN`

**Endpoint:** `POST /api/v3/openapi/temperate-token`

**Purpose:** Get temporary token (300 seconds duration)

**Request:**

```
{
  "duration": 300
}
```

**Response:**

```
{
  "code": 20000,
  "message": "success",
  "data": {
    "token": "Em2SB9ijEYrQimb0v8irW/WTOZm67dHHeqArhGBom9M="
  }
}
```

## 4. Get Exchange Token

**Command:** `GET_EXCHANGE_TOKEN`

**Endpoint:** `POST /api/v3/openapi/auth/exchange-identity`

**Purpose:** Exchange temporary token for identity token

**Prerequisites:** Must run GET_TEMP_TOKEN first.

**Response:**

```
{
  "code": 20000,
  "message": "success",
  "data": {
    "data": "Z6jSr30H6P60joVp69nGgTP8pvY4tEodneDAQY6Fe94="
  }
}
```

### 5. Get Devices

**Command:** `GET_DEVICES`

**Endpoint:** `GET /api/v3/openapi/devices-v2`

**Purpose:** List all devices for authenticated user

**Prerequisites:** Must run Login/Register first (needs Auth Token).

**Headers:**

```
Authorization: Bearer <auth_token>
```

### 6. Test Main Stream

**Command:** `TEST_MAIN_STREAM`

**Format:** `rtsp://<ip>:554/streamtype=1`

```
Main Stream RTSP URL: rtsp://192.168.1.100:554/streamtype=1
```

### 7. Test Sub Stream

**Command:** `TEST_SUB_STREAM`

**Format:** `rtsp://<ip>:554/streamtype=0`

```
Sub Stream RTSP URL: rtsp://192.168.1.100:554/streamtype=0
```

### 8. Get Snapshot URL

**Command:** `GET_SNAPSHOT_URL`

**Format:** `http://[user:pass@]<ip>:<port>/snap.jpg`

```
Snapshot URL: http://admin:password@192.168.1.100:80/snap.jpg
```

---

# Properties

## Input Properties (Configure These)

| Property | Type | Default | Description |
|---|---|---|---|
| Base API URL | STRING | https://api.arpha-tech.com | API endpoint |
| Account | STRING | — | User email or phone |
| IP Address | STRING | 192.168.1.100 | Camera IP |
| HTTP Port | INTEGER | 80 | HTTP port |
| RTSP Port | INTEGER | 554 | RTSP port |
| Username | STRING | — | Camera username |
| Password | STRING | — | Camera password |
| Snapshot Path | STRING | /snap.jpg | Snapshot path |
| Enable MQTT | LIST | True | Enable/disable MQTT event streaming |

## Output Properties (Read-Only/Managed)

| Property | Description |
|---|---|
| Status | Current operation status |
| Public Key | RSA public key from API |
| Client ID | Generated client ID |
| Auth Token | Authentication token |

| Property | Description |
| --- | --- |
| Temp Token | Temporary token |
| Exchange Token | Identity token |
| Main Stream URL | High quality RTSP URL |
| Sub Stream URL | Low quality RTSP URL |
| Online | Camera online status |

---

# Workflow Examples

## Workflow 1: Initial Setup & Authentication

```
1. Set "Base API URL" property
2. Set "Account" property (your email)
3. Execute "Initialize"
   → Gets public key
4. Execute "Login/Register"
   → Authenticates user
   → Stores Auth Token
   → MQTT auto-connects after auth
5. Execute "Get Devices"
   → Lists your devices
```

## Workflow 2: Token Management

```
1. Execute "Get Temp Token"
   → Gets 300-second token
2. Execute "Get Exchange Token"
   → Exchanges for identity token
3. Use tokens for custom API calls
```

## Workflow 3: Streaming Setup

```
1. Set "IP Address" property
2. Execute "Test Main Stream"
   → rtsp://192.168.1.100:554/streamtype=1
```

```
3. Execute "Test Sub Stream"
   → rtsp://192.168.1.100:554/streamtype=0
4. Use URLs in Control4 video configuration
   or external VMS
```

### Workflow 4: Snapshot Testing

```
1. Set "IP Address", "Username", "Password"
2. Execute "Get Snapshot URL"
3. URL sent to camera proxy
4. Test URL in browser to verify
```

### Workflow 5: Push Notifications

```
1. Complete Workflow 1 (auth + MQTT connected)
2. Create notification in Agents → Push Notification
3. Set Attachment Type = Snapshot URL
4. In Programming, map camera event → Send Push Notification
5. Trigger event to receive mobile notification with snapshot
```

---

## RSA Encryption

The driver uses **Control4's C4:Crypto() API** for RSA-OAEP + SHA256 encryption:

```
-- Create RSA crypto object
local crypto = C4:Crypto("RSA")

-- Import public key (from Initialize)
crypto:ImportPublicKey(publicKey)

-- Encrypt credentials
local encrypted = crypto:Encrypt(data, {
    scheme = "oaep",
    hash = "sha256"
})
```

### Requirements

- Control4 OS 3.x or newer

- Public key from Initialize API
- PEM formatted public key

### Fallback Mode

If `C4:Crypto()` is unavailable:

1. Set "Pre-Encrypted Post Data" property
2. Driver will use pre-encrypted value
3. For testing only

---

## Troubleshooting

### View Logs

1. Open **Composer Pro**
2. Go to **Lua Output** window
3. Execute any action
4. Watch detailed logs: request URLs, headers, bodies, response codes, and error messages

### Common Issues

#### "No public key available"

- Run Initialize first
- Check Base API URL
- Verify network connectivity

#### "Failed to create C4:Crypto object"

- Control4 OS too old (need 3.x+)
- Use fallback pre-encryption mode

#### "Login failed"

- Verify Account property is set
- Check public key exists
- Ensure Initialize ran successfully

**"IP Address not set"**

- Configure IP Address property
- Verify camera is reachable

**"No auth token available"**

- Run Login/Register first
- Check authentication succeeded

**MQTT not receiving events**

- Verify "Enable MQTT" is set to True
- Confirm Login/Register completed successfully
- Check motion detection is enabled in CldBus App
- Verify network allows outbound port 8884 (MQTT over SSL)

### API Response Codes

| Code | Meaning |
|---|---|
| 200 / 20000 | Success |
| 401 | Unauthorized (invalid token) |
| 400 | Bad request (invalid parameters) |
| 500 | Server error |

---

## Files Structure

```
Smart-Camera-K15-SL/
├── driver.lua              # Main driver logic
├── driver.xml              # Configuration & metadata
├── mqtt_manager.lua        # MQTT event streaming & broker management
├── README.md               # This documentation
├── CldBusApi/              # API helper libraries
│   ├── auth.lua
│   ├── dkjson.lua
│   ├── http.lua
│   ├── sha256.lua
│   ├── transport_c4.lua
```

```
│   └── util.lua
└── build-c4z.ps1           # Build script
```

# Building the Driver

## Build Package

Run in PowerShell:

```
.\build-c4z.ps1
```

This creates `Smart-Camera-K15-SL-v0.1.0.c4z` ready for installation.

## Manual Build

```
$files = @(
    "driver.lua",
    "driver.xml",
    "mqtt_manager.lua",
    "README.md",
    "CldBusApi\*.lua"
)

Compress-Archive -Path $files -DestinationPath "temp.zip"
Rename-Item "temp.zip" "Smart-Camera-K15-SL.c4z"
```

# Development Notes

## Helper Libraries (CldBusApi/)

- **auth.lua** — Authentication utilities
- **dkjson.lua** — JSON encoding/decoding
- **http.lua** — HTTP request helpers
- **sha256.lua** — HMAC-SHA256 implementation
- **transport_c4.lua** — Control4 transport adapter
- **util.lua** — UUID generation, HMAC functions

### Signature Generation

All API requests include HMAC-SHA256 signatures:

```
-- Message format
local message = "client_id=<uuid>&request_id=<uuid>&time=<timestamp>&version=0.0.1"

-- Generate signature
local signature = util.hmac_sha256_hex(message, app_secret)

-- App secret
local app_secret = "hg4IwDpf2tvbVdBGc6nwP5x2XGCIlNv8"
```

### UUID Generation

```
local client_id = util.uuid_v4()
-- Example: "0ffb133a-b03d-4cfd-82a9-d45220516199"
```

---

## Version History

### v10 (Current)

- K15 new UI.

### v9

- Added Motion Detection, Recording, Video Settings, and Local Storage in K15 Device Settings.

### v8

- Added face list feature.
- Added snapshot feature.
- Fix MQTT bug.

### v7

- Added restart functionality.

### v6

- Added firmware info.

### v4

- Auto-populate IP Address, VID and MAC Address.

### v3

- Implemented functionality to store all incoming events in the database.

### v2

- Fixed version display showing 0 in Control4 Manage Drivers interface
- Fixed device specific conditionals labels not showing in Control4 Composer

### v1.2

- Removed hardcoded AppName

### v1.1

-

### v0.1.0

- Complete API integration
- RSA-OAEP encryption with C4:Crypto()
- User authentication
- Token management
- Device listing
- RTSP streaming URLs
- Snapshot support
- PTZ controls
- MQTT real-time event detection (SSL, port 8884)
- Push notification support with snapshot attachment

---

## Support

- **Driver Version:** 10
- **Maintainer:** Slomins
- **Manufacturer:** Slomins
- **Model:** K15-SL
- **Control4 OS:** 3.3.2+
- **API Endpoint:** https://api.arpha-tech.com
- **Website:** www.slomins.com

For issues or questions, check Lua Output logs for detailed error information.

---

## License

Copyright © 2025 Slomins. All Rights Reserved.

---

*End of Documentation*